

View Only

QUESTIONS

Q1

Default and Parameterized Constructors
20 marks

Q2

Constructor Overloading
20 marks

Q3

Copy Constructor with Dynamic Memory
20 marks

Q4

Destructor and Resource Management
20 marks

Q5

Overloading the + Operator (Complex Numbers)
20 marks

5/5 answered

Q1 Default and Parameterized Constructors

20 marks

Write a C++ program that defines a class `Book` with attributes `title`, `author`, and `price`. Implement:

- A. A default constructor that initializes all attributes to default values ("Unknown", "Unknown", 0.0).
- B. A parameterized constructor that accepts values for all three attributes.
- C. A `display()` member function to print the book details.

In `main()`, create one object using each constructor and display its details.

Your Code

```
1 #include<iostream>
2 #include<string>
3 using namespace std;
4 class Book{
5     string title,author;
6     double price;
7
8     public:
9     Book(){
10         title="Unknown";
11         author="Unknown";
12         price=0.0;
13     }
14
15     Book(string t, string a, double p){
16         title=t;
17         author=a;
18         price=p;
19     }
20
21     void display(){
22         cout<<"Title: "<<title<<endl;
23         cout<<"Author: "<<author<<endl;
24         cout<<"Price: "<<price<<endl;
25     }
26 };
27
28 int main(){
29     Book b1;
30     cout<<"Default constructor: "<<endl;
31     b1.display();
32
33     cout<<endl;
34
35     Book b2("Harry Potter","J.K.Rowling",599.50);
36     cout<<"Parameterized Constructor: "<<endl;
37     b2.display();
38
39     return 0;
40 }
```

Input

stdin input

Output

Visible Test Cases

Test Case 1

Test Case 2

Test Case 3

QUESTIONS

Q1

Default and Parameterized
Constructors
20 marks

Q2

Constructor Overloading
20 marks

Q3

Copy Constructor with Dynamic
Memory
20 marks

Q4

Destructor and Resource
Management
20 marks

Q5

Overloading the + Operator
(Complex Numbers)
20 marks

5/5 answered

Q2 Constructor Overloading

20 marks

Write a C++ program that defines a class Rectangle with the following constructors:

A. A default constructor that sets length and breadth to 1.

B. A single-parameter constructor that creates a square (length = breadth).

C. A two-parameter constructor that sets both length and breadth.

Include a member function area() to compute the rectangle's area. In main(), create three objects using each constructor and print their areas.

Your Code

```
1 #include<iostream>
2 using namespace std;
3 class Rectangle{
4     int length, breadth;
5
6     public:
7     Rectangle(){
8         length=1;
9         breadth=1;
10    }
11    Rectangle(int x){
12        length=x;
13        breadth=x;
14    }
15    Rectangle(int l, int b){
16        length=l;
17        breadth=b;
18    }
19    int area(){
20        return length*breadth;
21    }
22 };
23
24 int main(){
25     Rectangle r1;
26     Rectangle r2(5);
27     Rectangle r3(6,4);
28
29     cout<<"Area of default rectangle: "<<r1.area()<<endl;
30     cout<<"Area of square: "<<r2.area()<<endl;
31     cout<<"Area of rectangle: "<<r3.area()<<endl;
32
33     return 0;
34 }
```

Input

stdin input

Output

Visible Test Cases

Test Case 1

Test Case 2

Test Case 3

View Only

QUESTIONS

Q1

Default and Parameterized Constructors
20 marks

Q2

Constructor Overloading
20 marks

Q3

Copy Constructor with Dynamic Memory
20 marks

Q4

Destructor and Resource Management
20 marks

Q5

Overloading the + Operator (Complex Numbers)
20 marks

5/5 answered

Q3 Copy Constructor with Dynamic Memory

20 marks

Write a C++ program that defines a class Student containing a dynamically allocated character array for the student's name and an integer for marks. Implement:

- A. A parameterized constructor that allocates memory and copies the name.
- B. A deep copy constructor that allocates new memory for the copied object.
- C. A destructor that releases the allocated memory.
- D. A display() function to print the student's details.

In main(), create a Student object, create another object using the copy constructor, and modify the second object's name to demonstrate that the original remains unaffected.

Your Code

```
1 #include<iostream>
2 #include<string>
3 using namespace std;
4 class Student{
5     char* name;
6     int marks;
7
8     public:
9     Student(const char* n,int m){
10         name = new char[strlen(n)+1];
11         strcpy(name, n);
12         marks=m;
13     }
14
15     Student(const Student& s){
16         name=new char[strlen(s.name)+1];
17         strcpy(name,s.name);
18         marks=s.marks;
19     }
20     ~Student(){
21         delete[] name;
22     }
23
24     void setName(const char* n){
25         delete[] name;
26         name= new char[strlen(n)+1];
27         strcpy(name, n);
28     }
29
30     void display(){
31         cout<<"Name: "<<name<<endl;
32         cout<<"Marks: "<<marks<<endl;
33     }
34 };
35
36 int main(){
37     Student s1("John", 85);
38
39     Student s2=s1;
40
41     s2.setName("David");
42
43     cout<<"Original Student:"<<endl;
44     s1.display();
45     cout<<endl;
46     cout<<"Copied Student:"<<endl;
47     s2.display();
48
49     return 0;
50 }
```

Input

stdin input

View Only

QUESTIONS

Q1

Default and Parameterized
Constructors
20 marks

Q2

Constructor Overloading
20 marks

Q3

Copy Constructor with Dynamic
Memory
20 marks

Q4

Destructor and Resource
Management
20 marks

Q5

Overloading the + Operator
(Complex Numbers)
20 marks

5/5 answered

Q4 Destructor and Resource Management

20 marks

Write a C++ program that defines a class `FileHandler` which simulates opening and closing a file. Implement:

A. A constructor that prints "File <name> opened".

B. A destructor that prints "File <name> closed".

In `main()`, create three `FileHandler` objects inside a block `{ }` and observe the order in which constructors and destructors are invoked. Print a message after the block ends to confirm the destruction order.

Your Code

```
1 #include <iostream>
2 #include <string>
3 using namespace std;
4
5 class FileHandler {
6     string name;
7
8 public:
9     FileHandler(string n) {
10         name = n;
11         cout << "File " << name << " opened" << endl;
12     }
13
14     ~FileHandler() {
15         cout << "File " << name << " closed" << endl;
16     }
17 };
18
19 int main() {
20     {
21         FileHandler f1("File1");
22         FileHandler f2("File2");
23         FileHandler f3("File1");
24     }
25
26     cout << "Block ended. All files are closed." << endl;
27
28     return 0;
29 }
```

Input

stdin input

Output

Visible Test Cases

Test Case 1

Test Case 2

Test Case 3

View Only

QUESTIONS

Q1

Default and Parameterized Constructors
20 marks

Q2

Constructor Overloading
20 marks

Q3

Copy Constructor with Dynamic Memory
20 marks

Q4

Destructor and Resource Management
20 marks

Q5

Overloading the + Operator (Complex Numbers)
20 marks

5/5 answered

Q5 Overloading the + Operator (Complex Numbers)

20 marks

Write a C++ program that defines a class Complex with real and imaginary parts. Overload the + operator to add two complex numbers and return a new Complex object. Include a display() function to print the result in the form a + bi. In main(), create two Complex objects, add them using the overloaded + operator, and display the result.

Your Code

```
1 #include<iostream>
2 using namespace std;
3 class Complex{
4     private:
5         double real;
6         double imag;
7
8     public:
9         Complex(double r=0.0, double i=0.0){
10             real = r;
11             imag = i;
12         }
13         Complex operator+(const Complex& obj){
14             Complex temp;
15             temp.real = this->real + obj.real;
16             temp.imag = this->imag + obj.imag;
17             return temp;
18         }
19
20         void display()const{
21             if(imag >= 0){
22                 cout<< real << " + " << imag << "i" << endl;
23             }
24             else{
25                 cout<<real<< " - " << imag << "i" << endl;
26             }
27         }
28 };
29
30 int main(){
31     Complex c1(3.5,2.5);
32     Complex c2(1.2,4.3);
33
34     Complex c3= c1+c2;
35     c3.display();
36
37     return 0;
38 }
```

Input

stdin input

Output

Visible Test Cases

Test Case 1

Test Case 2

Test Case 3